Software Engineering – Unit 2

Complete Semester Exam Notes | 6-Mark Answers

Syllabus covered:

- Software Requirement Analysis and Specification
- Need for SRS
- Characteristics of Requirements
- Functional Requirements
- Non-Functional Requirements
- SRS Document – IEEE Format
- Function-Oriented Design
- Structured Design Methodology
- Data Flow Diagram (DFD)
- Design Principles
- Module-Level Concepts
- Cohesion and Coupling

The notes below use simple English, clear headings, examples, and exam-ready explanations suitable for six-mark answers.

1. Software Requirement Analysis and Specification (SRS)

Definition

Software Requirement Analysis and Specification is the process of understanding what the customer needs and documenting those requirements in a formal document called the **Software Requirements Specification (SRS)**.

1. Requirement Analysis

Requirement analysis is the process of collecting, studying, and understanding the customer's needs.

Steps involved:

1. Requirement Gathering: Collect information from customers and users.

2. Requirement Analysis: Study the collected requirements and identify problems or conflicts.

3. Requirement Prioritization: Identify which requirements are most important.

4. Requirement Validation: Check whether the requirements are correct, complete, and consistent.

5. Requirement Documentation: Record the requirements in the SRS document.

2. Requirement Specification

Requirement specification is the process of documenting the analyzed requirements clearly so that developers and customers understand what the software must do.

It includes:

- Functional requirements.

- Non-functional requirements.
- System constraints.
- External interface requirements.
- Performance and security requirements.

Example: Online Shopping System

Suppose we are developing an online shopping application. The requirements might include:

- Users must be able to register and log in.
- Users must be able to search for products.
- Users must be able to add products to their carts.
- The system must process payments securely.
- The system must display search results within two seconds under specified conditions.

These requirements are analyzed and documented in the SRS.

Advantages

1. Provides a clear understanding of customer needs.

2. Reduces misunderstandings between customers and developers.

3. Helps estimate development cost and time.

4. Supports software design and testing.

5. Reduces the risk of project failure.

Conclusion: Requirement analysis identifies what the customer needs, while requirement specification documents those needs in a structured form.

2. Need for SRS

Definition

An **SRS (Software Requirements Specification)** is a formal document that describes the functional and non-functional requirements of a software system. It acts as an agreement between the customer and the development team regarding what the software should do.

Why Do We Need an SRS?

1. Clear Understanding of Requirements: An SRS clearly explains the expected behavior and features of the software.

2. Communication Between Customer and Developer: It provides a common reference for customers, developers, testers, and project managers.

3. Helps in Software Design: Developers use the SRS to determine the system architecture, modules, and interfaces.

4. Supports Testing: Testers use the requirements to prepare test cases and verify whether the software meets the specified requirements.

5. Cost and Time Estimation: The development team can estimate the resources, budget, and time required for the project.

6. Reduces Development Errors: A well-prepared SRS helps identify missing, conflicting, or unclear requirements before implementation begins.

7. Supports Maintenance: When changes are required, developers can refer to the SRS to understand the system's expected behavior.

Example

Consider a college management system. Its SRS specifies that:

- Students can view attendance.
- Faculty can update attendance.
- Administrators can manage student records.
- Only authorized users can access confidential information.

Developers use these requirements to build the system, while testers verify that the features work correctly.

Conclusion: SRS is essential because it provides a clear, documented foundation for software development, testing, and maintenance.

3. Characteristics of Good Requirements

Definition

Good software requirements must possess certain characteristics to ensure that the software can be developed and tested successfully.

Important Characteristics

1. Correct: Each requirement must represent an actual customer need. *Example:* A banking system should calculate account balances correctly.

2. Complete: All necessary requirements must be included. *Example:* An online shopping system should specify product selection, payment, order confirmation, and order tracking.

3. Unambiguous: Every requirement should have only one possible interpretation. *Incorrect:* “The system should respond quickly.” *Correct:* “The system should display search results within two seconds under specified operating conditions.”

4. Consistent: Requirements must not contradict one another. *Example:* One requirement must not state that users can access accounts without logging in while another requires authentication for every account operation.

5. Verifiable (Testable): It must be possible to check whether a requirement has been satisfied. *Example:* The system must support 1,000 simultaneous users under the specified test conditions.

6. Feasible: The requirements must be achievable using available technology, budget, and time.

7. Necessary: Every requirement must contribute to a genuine system need.

8. Traceable: Each requirement should have a unique identifier so that it can be tracked through design, implementation, and testing.

9. Modifiable: Requirements should be organized so that changes can be made without creating unnecessary confusion.

Conclusion: A good requirement should be correct, complete, unambiguous, consistent, verifiable, feasible, necessary, traceable, and modifiable.

Exam tip: If the question asks for six marks, write the definition and explain at least six characteristics with examples.

4. Functional Requirements

Definition

Functional requirements describe **what the software system must do**. They specify the services, operations, and behavior that the system must provide.

Important Functional Requirements

1. Input Processing: The system must accept and process user input. *Example:* A student enters a hall ticket number to view examination results.

2. Data Processing: The system must perform the required calculations or operations. *Example:* A banking system calculates the account balance after a withdrawal.

3. Output Generation: The system must produce the required output. *Example:* A college system displays a student's semester results.

4. User Authentication: The system must verify user credentials when authentication is required. *Example:* A user logs in using a username and password.

5. Data Storage and Retrieval: The system must store and retrieve information. *Example:* A library system stores book details and retrieves them when requested.

6. Business Rules: The system must enforce rules that govern its operations. *Example:* A library system must prevent issuing a book that is unavailable.

Example: Online Shopping System

Its functional requirements include:

1. The system shall allow customers to register.

2. The system shall allow customers to log in.

3. The system shall allow customers to search for products.

4. The system shall allow customers to add products to a cart.

5. The system shall process orders and payments.

6. The system shall generate order confirmation.

Importance

- Defines the required software features.
- Guides software design and development.
- Helps testers prepare test cases.
- Ensures the software meets customer needs.

Conclusion: Functional requirements specify the operations and services that a software system must perform.

Remember: Functional requirement = **What the system does.**

5. Non-Functional Requirements

Definition

Non-functional requirements specify **how well the software must operate** and the constraints under which it must work. They describe quality attributes such as performance, security, reliability, and usability.

Types of Non-Functional Requirements

1. Performance: Specifies response time, throughput, and resource usage. *Example:* The system should display search results within two seconds under specified conditions.

2. Reliability: Specifies the ability of the system to perform correctly and consistently. *Example:* A banking system should process transactions accurately and recover appropriately from failures.

3. Security: Specifies how the system protects information against unauthorized access. *Example:* Only authorized users can access confidential student records.

4. Usability: Specifies how easy the system is to learn and use. *Example:* Users should be able to complete registration through a simple, clearly labeled form.

5. Maintainability: Specifies how easily the software can be corrected, modified, or improved. *Example:* The system should use modular code so that individual features can be updated independently.

6. Portability: Specifies how easily software can operate in different environments. *Example:* A web application should support the specified versions of major browsers.

7. Scalability: Specifies the system's ability to handle increasing workloads. *Example:* An e-commerce system should accommodate growth in users and transactions.

8. Availability: Specifies the proportion of time the system should remain operational. *Example:* An online service may be required to achieve 99.9% monthly availability.

Example: Banking Application

Requirement	Type
Users can transfer money	Functional
Transactions must be processed securely	Non-functional
Users can view account balances	Functional
The system must meet a specified response time	Non-functional
The system must meet a defined availability target	Non-functional

Importance

1. Improves software quality.

2. Increases user satisfaction.

3. Helps select appropriate technologies and architecture.

4. Supports performance and security testing.

5. Helps establish measurable quality targets.

Conclusion: Non-functional requirements define the quality attributes and operating constraints of software.

Remember: Non-functional requirement = **How well the system performs.**

Difference Between Functional and Non-Functional Requirements

Functional Requirements	Non-Functional Requirements
Define what the system does.	Define how well the system operates.
Describe features and services.	Describe quality attributes and constraints.
Usually concern specific operations.	Often apply across multiple system operations.
Example: User can log in.	Example: Login must respond within two seconds.
Tested using functional test cases.	Tested using performance, security, usability, or other quality tests.

6. SRS Document – IEEE Format

Definition

An SRS document describes the complete requirements of a software system in a structured and organized format. The IEEE SRS format provides a standard structure for documenting software requirements.

In exams, this topic is commonly answered using the traditional IEEE 830-style organization. Modern projects may use newer standards or organizational templates.

Structure of an SRS Document

1. Introduction: Provides an overview of the software. It contains purpose (why the document is prepared), scope (what the software will and will not do), definitions and abbreviations (meaning of technical terms), references (documents used while preparing the SRS), and overview (organization of the document).

2. Overall Description: Explains the general characteristics of the software. It includes product perspective, product functions, user characteristics, general constraints, assumptions, and dependencies.

3. Specific Requirements: Contains detailed system requirements. It includes functional requirements, external interface requirements, performance requirements, database requirements, design constraints, software quality attributes, and other non-functional requirements.

4. Appendices: Contains supporting information, such as additional tables, diagrams, or technical details.

5. Index: May be included to help readers locate information quickly.

Example: Library Management System

An SRS for a library system may contain:

- **Introduction:** Purpose and scope of the library system.
- **Overall Description:** Intended users, such as librarians and students.
- **Functional Requirements:** Search books, issue books, and return books.
- **Non-Functional Requirements:** Security, performance, and reliability targets.
- **External Interfaces:** User interface and database interfaces.
- **Appendices:** Supporting diagrams and tables.

Advantages of the IEEE Format

1. Provides a systematic structure for requirements.

2. Improves readability and understanding.

3. Helps developers and testers locate requirements.

4. Reduces ambiguity and omissions.

5. Supports requirement verification and maintenance.

Conclusion: The IEEE-style SRS format organizes software requirements into clear sections, helping stakeholders understand, develop, and test the system.

Exam tip: For this question, draw a tree diagram showing the major SRS sections if time permits.

7. Function-Oriented Design

Definition

Function-oriented design is a software design approach in which a system is divided into smaller functions or modules, with each module performing a specific task. It focuses on **what operations the system performs and how data flows between those operations.**

Main Concepts

1. Functional Decomposition: A large system is divided into smaller functions. *Example:* An online shopping system can be divided into login, product search, cart management, payment, and order processing.

2. Data Flow: The movement of input data through different functions is analyzed. *Example:* Customer details flow from a registration form to a validation function and then to the database.

3. Modularity: Each function is designed as a separate module to simplify development and testing.

4. Structured Design: The system structure is developed using tools such as Data Flow Diagrams and structure charts.

5. Cohesion and Coupling: Modules should have high cohesion and low coupling for better maintainability.

Steps in Function-Oriented Design

1. Study and understand the requirements.

2. Create a Data Flow Diagram (DFD).

3. Identify the major functions.

4. Divide major functions into smaller modules.

5. Define relationships between modules.

6. Evaluate module cohesion and coupling.

7. Prepare the final design and structure chart.

Example: Student Result Processing System

The system can be divided into these functions:

- Read student details.
- Accept marks.
- Calculate total marks.
- Calculate percentage.
- Determine grade.
- Display results.

Each function performs a specific task and interacts with other functions through defined inputs and outputs.

Advantages

1. Reduces system complexity.

2. Supports modular development.

3. Simplifies testing and debugging.

4. Encourages code reuse.

5. Improves maintainability.

Limitations

1. Changes in shared data can affect multiple functions.

2. Large systems may become difficult to manage if dependencies are excessive.

3. It focuses primarily on functions rather than representing real-world entities as objects.

Conclusion: Function-oriented design simplifies software development by decomposing a system into well-defined functions and modules.

8. Structured Design Methodology

Definition

Structured design is a systematic software design methodology that transforms the results of requirement analysis into a modular software architecture. It commonly uses **Data Flow Diagrams (DFDs), structure charts, and functional decomposition** to identify system modules.

Steps in Structured Design

1. Study the SRS: Understand the functional requirements and expected system behavior.

2. Develop the DFD: Represent how data enters the system, gets processed, is stored, and leaves the system.

3. Identify Major Functions: Identify the main processes from the DFD.

4. Perform Functional Decomposition: Break large processes into smaller, manageable functions.

5. Prepare a Structure Chart: Represent the hierarchy of modules and the relationships between them.

6. Evaluate Cohesion and Coupling: Aim for high cohesion within modules and low coupling between modules.

7. Refine the Design: Improve the structure to make the modules easier to understand, test, and maintain.

Example: ATM System

A structured design for an ATM may contain the following modules:

- Authentication.
- Account selection.
- Balance inquiry.
- Cash withdrawal.
- Cash deposit.
- Transaction receipt.

The cash withdrawal module may be divided further into:

1. Accept withdrawal amount.

2. Validate the amount.

3. Check account balance.

4. Dispense cash.

5. Update the account.

6. Print the receipt.

Advantages

- Provides a systematic design process.
- Improves modularity.
- Simplifies implementation and testing.
- Makes the system easier to maintain.
- Helps identify module responsibilities.

Conclusion: Structured design converts requirements into a clear hierarchy of software modules using DFDs and structure charts.

9. Data Flow Diagram (DFD)

Definition

A Data Flow Diagram (DFD) is a graphical representation of how data moves through a system, including its sources, destinations, processing activities, and data stores. A DFD focuses on **the flow and transformation of data**, rather than the sequence of program instructions.

Symbols Used in a DFD

There are four main components:

Symbol	Name	Meaning
Rectangle	External Entity	A person or external system that sends or receives data.
Circle or Rounded Rectangle	Process	Transforms input data into output data.
Arrow	Data Flow	Shows the direction of data movement.
Two Parallel Lines or Open Rectangle	Data Store	Represents stored data.

Note: Exact symbol shapes may vary between DFD notations.

Levels of DFD

1. Context Diagram (Level 0 in many conventions)

It shows the entire system as one process and its interactions with external entities. Example: A library management system interacts with students and librarians.

2. Level 1 DFD

It divides the main system into major processes. For a library system, the processes may include:

- Search books.
- Issue books.
- Return books.
- Maintain book records.

3. Level 2 DFD

It breaks a Level 1 process into more detailed subprocesses. For example, the issue-book process can be divided into:

1. Receive student ID and book ID.

2. Validate student details.

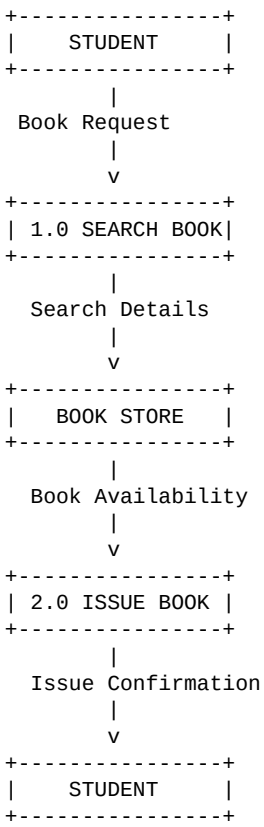
3. Check book availability.

4. Record the issue.

5. Confirm the transaction.

Example: Library Management System DFD

Consider a simplified system in which a student requests a book.



This is a simplified conceptual illustration. In a formal DFD, the data store should use its designated symbol, and the diagram should clearly distinguish the data flows associated with each process.

Rules for Drawing a DFD

1. Every process should have at least one input and one output.

2. Data should flow through a process when it is transformed.

3. External entities should not directly exchange data with data stores.

4. External entities should not directly exchange data with one another within the system's DFD.

5. Each data flow should have a meaningful name.

6. Lower-level DFDs should preserve the inputs and outputs of their parent processes. This is called **balancing**.

Advantages of DFD

- Easy to understand.
- Shows how data moves through the system.
- Helps identify major processes and data stores.
- Supports communication between analysts and customers.
- Helps in functional decomposition and structured design.

Conclusion: A DFD represents data movement and processing within a system and is an important tool in requirement analysis and structured design.

10. Design Principles in Software Engineering

Definition

Software design principles are guidelines used to create software that is understandable, maintainable, reliable, and easy to modify.

Important Design Principles

1. Modularity: Divide the software into smaller modules, each with a specific responsibility. *Example:* Separate login, payment, and reporting modules.

2. Abstraction: Focus on essential features while hiding unnecessary implementation details. *Example:* A user clicks a payment button without needing to understand the internal payment-processing algorithm.

3. Information Hiding: Hide the internal implementation details of a module and expose only the necessary interface. *Example:* A banking module provides a transfer operation without exposing its internal balance-update logic.

4. Separation of Concerns: Separate different responsibilities so that each part addresses a particular concern. *Example:* Keep user interface code separate from database access code.

5. High Cohesion: A module should perform closely related tasks. *Example:* A payment module should handle payment-related operations rather than unrelated user-profile operations.

6. Low Coupling: Modules should have as few unnecessary dependencies on one another as possible. *Example:* A notification module should receive the information it needs through a clear interface instead of directly modifying order records.

7. Simplicity: Avoid unnecessary complexity and design only what is needed.

8. Reusability: Design components so they can be reused in other parts of the system or in other projects where appropriate.

9. Maintainability: The design should make it easy to locate, correct, and improve software components.

10. Traceability: Design elements should be traceable to the requirements they implement.

Importance

1. Improves software quality.

2. Reduces maintenance effort.

3. Simplifies testing and debugging.

4. Supports team development.

5. Reduces the impact of changes.

Conclusion: Good software design follows principles such as modularity, abstraction, information hiding, high cohesion, low coupling, and simplicity.

11. Module-Level Concepts

Definition

Module-level concepts describe how software is divided into individual modules and how each module interacts with other modules. A module is a separate part of a software system that performs a particular task.

Important Module-Level Concepts

1. Modularity: A software system is divided into smaller modules. *Example:* An e-commerce system contains product, cart, payment, and order modules.

2. Module Interface: The interface defines how other modules communicate with a module. *Example:* A payment module may accept an order ID and payment amount and return a payment status.

3. Information Hiding: A module hides its internal implementation and exposes only the necessary interface.

4. Functional Independence: Each module should perform a clearly defined task with minimal dependence on other modules. Functional independence is achieved through high cohesion and low coupling.

5. Cohesion: Cohesion measures how strongly the elements within a module are related to one another. High cohesion is desirable.

6. Coupling: Coupling measures the degree of dependency between different modules. Low coupling is desirable.

Example

Consider a college management system. It contains:

- Student module.
- Attendance module.
- Examination module.
- Fee payment module.

Each module has a separate responsibility. The modules communicate through defined interfaces rather than unnecessarily accessing one another's internal details.

Advantages of Good Module Design

1. Simplifies development.

2. Makes testing easier.

3. Improves reusability.

4. Reduces the effect of changes.

5. Improves maintainability.

Conclusion: Good module-level design creates independent, well-defined components with clear interfaces, high cohesion, and low coupling.

12. Cohesion and Coupling

This is one of the most important topics in your Unit 2 syllabus. Prepare both definitions, types, examples, and the difference between them.

A. Cohesion

Definition

Cohesion is the degree to which the elements within a module are related to one another and contribute to a common purpose. **High cohesion is desirable** because it makes a module easier to understand, test, and maintain.

Types of Cohesion

The following types are generally arranged from weakest to strongest.

1. Coincidental Cohesion: Unrelated tasks are grouped into one module without a meaningful relationship. *Example:* A module calculates salaries, prints reports, and validates passwords.

2. Logical Cohesion: Tasks of the same general category are grouped together, and a control parameter determines which task is performed. *Example:* A module handles different types of input operations, such as keyboard input and file input.

3. Temporal Cohesion: Tasks are grouped because they occur at the same time. *Example:* A startup module loads configuration, opens log files, and initializes connections.

4. Procedural Cohesion: Tasks are grouped because they must follow a particular sequence, even though they may operate on different data. *Example:* A module validates an input, records an event, and displays a message in a required order.

5. Communicational Cohesion: Tasks are grouped because they operate on the same data. *Example:* A module retrieves a student's record and calculates a result using that record.

6. Sequential Cohesion: The output of one task becomes the input of the next task within the same module. *Example:* A module reads marks, calculates the total, and then calculates the percentage.

7. Functional Cohesion: All elements of a module contribute to one well-defined function. *Example:* A module performs only the calculation of a student's final grade. Functional cohesion is generally considered the strongest form of cohesion in the traditional classification.

Advantages of High Cohesion

- Easier to understand.
- Easier to test.
- Easier to maintain.
- Improves reusability.
- Reduces unrelated responsibilities within a module.

B. Coupling

Definition

Coupling is the degree of interdependence between different software modules. **Low coupling is desirable** because changes in one module are less likely to affect other modules.

Types of Coupling

The following list gives the traditional classification from strongest or most undesirable to weakest or most desirable.

1. Content Coupling: One module directly accesses or modifies the internal code or data of another module. *Example:* Module A directly changes a local variable inside Module B. This is generally the most undesirable form.

2. Common Coupling: Multiple modules share the same global data. *Example:* Several modules directly read and modify a common global variable storing the current user.

3. External Coupling: Modules depend on externally imposed formats, protocols, or device interfaces. *Example:* Two modules depend on the same externally defined file format.

4. Control Coupling: One module passes control information that determines the behavior of another module. *Example:* A function receives a flag that determines whether it should perform a deposit or a withdrawal.

5. Stamp Coupling: A module passes a complete data structure to another module even though the receiving module needs only part of it. *Example:* A function receives an entire student record but uses only the student's ID.

6. Data Coupling: Modules communicate by passing only the necessary data through their interfaces. *Example:* A grade-calculation module receives only the marks needed to calculate a grade. Data coupling is generally desirable in the traditional classification.

Note: Some textbooks include message coupling, which is commonly associated with object-oriented systems, as another low-coupling form.

Difference Between Cohesion and Coupling

Cohesion	Coupling
Measures relationships within a module.	Measures dependencies between modules.
Focuses on internal module organization.	Focuses on interactions between modules.
High cohesion is desirable.	Low coupling is desirable.
Improves module clarity and independence of purpose.	Reduces the impact of changes between modules.
Example: A module only calculates grades.	Example: A grade module receives marks through a clear interface.

Easy Example to Remember

Consider a college management system.

High Cohesion: The attendance module handles attendance-related operations only.

Low Coupling: The attendance module communicates with the student module through a defined interface rather than directly changing its internal data.

A well-designed system aims for **high cohesion and low coupling.**

Conclusion: Cohesion measures how closely related the responsibilities within a module are, whereas coupling measures the dependency between modules. Good software design aims for high cohesion and low coupling.

Quick Revision: Unit 2

Use this table to revise the complete syllabus before your examination.

Topic	Key Point to Remember
Requirement Analysis	Understand and analyze customer needs.
Requirement Specification	Document requirements clearly.
Need for SRS	Communication, design, testing, and maintenance.
Good Requirements	Correct, complete, unambiguous, consistent, verifiable, feasible, traceable, modifiable.
Functional Requirements	What the software does.
Non-Functional Requirements	How well the software operates.
IEEE SRS Format	Introduction, overall description, specific requirements, appendices, and index where applicable.
Function-Oriented Design	Divide the system into functions.
Structured Design	Convert requirements into a modular design.
DFD	Shows data sources, processes, stores, and flows.
Design Principles	Modularity, abstraction, information hiding, simplicity, and maintainability.
Module-Level Concepts	Modules, interfaces, functional independence, cohesion, and coupling.
Cohesion	Relationships within a module; high is desirable.
Coupling	Dependencies between modules; low is desirable.

Most Important Questions for the 6-Mark Exam

If you have limited preparation time, prioritize these questions:

1. Explain the need for SRS and the characteristics of a good SRS.

2. Differentiate functional and non-functional requirements with examples.

3. Explain the structure of an SRS document according to the IEEE format.

4. Explain function-oriented design and structured design methodology.

5. What is a DFD? Explain its symbols, levels, and rules with an example.

6. Explain the important software design principles.

7. Explain module-level concepts in software design.

8. Define cohesion and coupling. Explain their types with examples.

9. Differentiate cohesion and coupling.

10. Explain software requirement analysis and specification.

My recommendation: Prepare the DFD, IEEE SRS format, and cohesion-versus-coupling classifications particularly well. For a six-mark answer, aim to write a definition, six or more explained points, a relevant example or diagram where appropriate, and a short conclusion.